

RELATÓRIO PRÁTICO DE ATIVIDADE – ETAPA 6

Projeto Integrador: Assistente de Desenvolvimento de Sistemas

1. O Que a atividade pede:

Na Etapa 6 do Projeto Integrador, o objetivo principal foi preparar a aplicação que antes funcionava apenas em janelas no computador (Java Desktop com Swing) para que pudesse se transformar em um sistema que roda em navegadores de internet.

Para que não houvesse retrabalho, a atividade exigiu:

- Desacoplar as regras de negócio das telas visuais, impedindo que o código fique preso aos botões e caixas de texto do Swing.
- Aplicar princípios de boa organização de código (SOLID), em especial a **Responsabilidade Única (SRP)**.
- Eliminar *code smells* no código.
- Publicar a versão limpa do projeto em um novo repositório no GitHub e comprovar que tudo funciona por meio de testes práticos no método principal (main).

2. O Que Foi Feito e Para Que Serviu

Desenvolvido um projeto novo no NetBeans chamado SistemaEstoqueRefatorado, separando as funcionalidades em pastas (pacotes) organizadas como caixas com funções exclusivas:

- Classe Produto (pasta model): Serve exclusivamente como o molde do objeto e protege esses dados com métodos de acesso.
- Fábrica ProdutoFactory (pasta factory): Aplica o padrão de projeto *Factory Method*. Ela funciona como uma triagem que impede o cadastro de produtos com nomes em branco ou valores negativos antes mesmo de o objeto ser criado.
- Contrato e Persistência ProdutoRepository e ProdutoRepositoryMemoria (pasta repository): Cuidam unicamente de guardar, consultar e excluir produtos. Uma lista em memória para testes imediatos, mas a estrutura já está pronta para se conectar a um banco de dados real (MySQL/JPA) na fase web sem mexer nas regras de negócio (aplicando a Inversão de Dependência - DIP).
- Serviço de Negócios ProdutoService: Calcula descontos com limites de segurança e dá baixa no estoque apenas se houver quantidade suficiente, avisando o usuário caso ocorra algum erro.
- Testes no main (SistemaEstoqueRefatorado): Uma rotina de testes que simula na prática o cadastro de itens, o cálculo de descontos, a atualização de estoque e o bloqueio de operações inválidas sem depender de telas abertas.

3. Como o Bloco de Notas e o Prompt Foram Utilizados

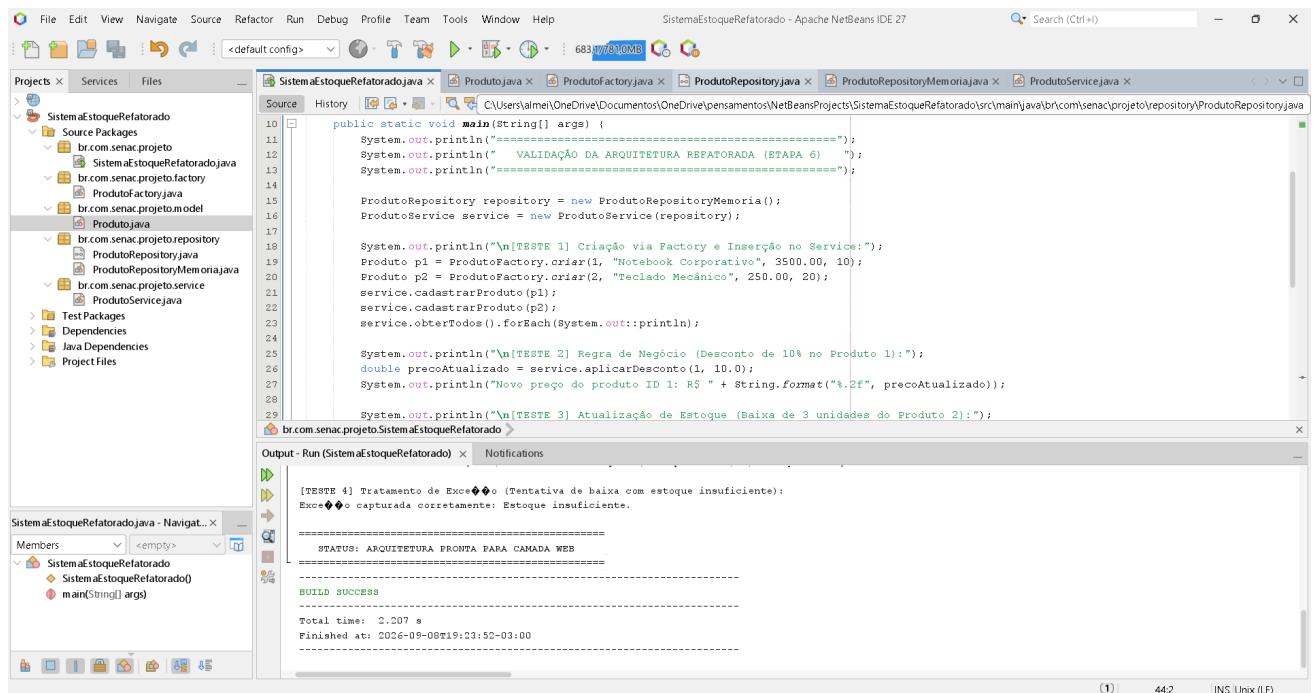
Para evitar o esforço físico e o desgaste de abrir terminais e digitar dezenas de linhas de comando repetitivas durante o processo de envio dos arquivos, utilizei uma rotina de automação com ferramentas simples do próprio computador:

- Bloco de Notas: Foi utilizado para redigir um roteiro de comandos (chamado script de lote ou arquivo .bat). Nele, escrevemos todas as instruções que o Git precisava seguir de uma só vez.
- Arquivo .gitignore: Criado via comandos para instruir o sistema a não enviar pastas temporárias ou arquivos pesados de compilação, mantendo apenas os códigos essenciais no repositório.
- Prompt de Comando: Ao dar dois cliques no arquivo gerado pelo Bloco de Notas, o terminal abriu sozinho e executou automaticamente o rastreamento dos arquivos, a criação do commit com a mensagem da etapa e o upload direto para a nuvem do GitHub.

Procurei rapidez, eliminando a margem de erros de digitação e enviando o repositório de forma limpa em poucos segundos.

4. Evidências de Validação

Execução dos Testes no Console:



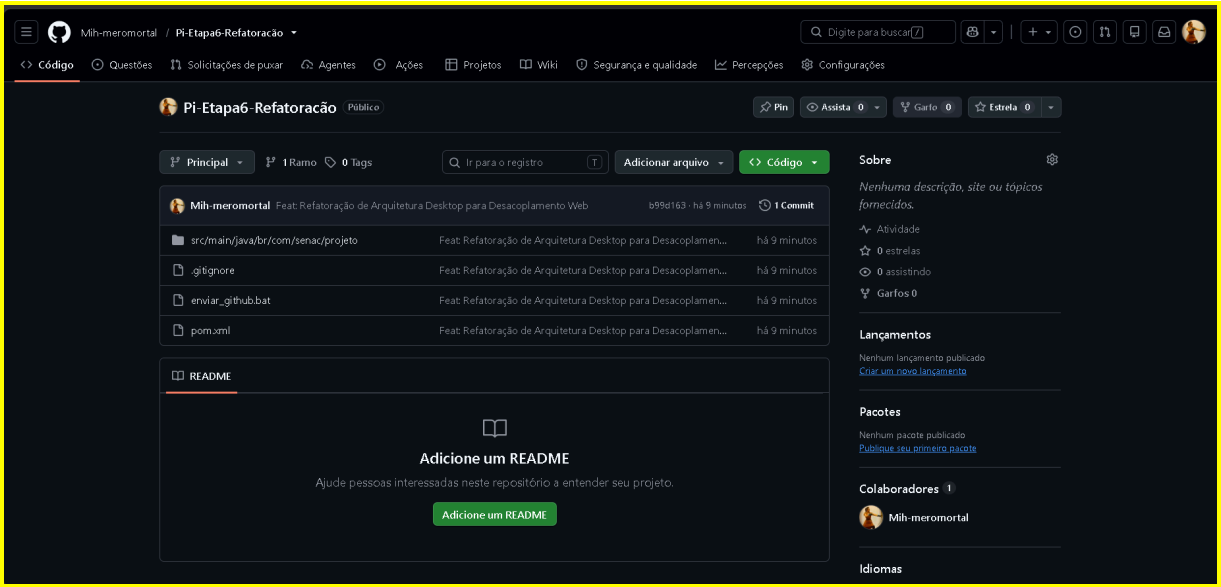
```

10 public static void main(String[] args) {
11     System.out.println("=====");
12     System.out.println("  VALIDAÇÃO DA ARQUITETURA REFATORADA (ETAPA 6)  ");
13     System.out.println("=====");
14
15     ProdutoRepository repository = new ProdutoRepositoryMemoria();
16     ProdutoService service = new ProdutoService(repository);
17
18     System.out.println("\n[TESTE 1] Criação via Factory e Inserção no Service:");
19     Produto p1 = ProdutoFactory.criar(1, "Notebook Corporativo", 3500.00, 10);
20     Produto p2 = ProdutoFactory.criar(2, "Teclado Mecânico", 250.00, 20);
21     service.cadastrarProduto(p1);
22     service.cadastrarProduto(p2);
23     service.obterTodos().forEach(System.out::println);
24
25     System.out.println("\n[TESTE 2] Regra de Negócio (Desconto de 10% no Produto 1):");
26     double precoAtualizado = service.aplicarDesconto(1, 10.0);
27     System.out.println("Novo preço do produto ID 1: R$ " + String.format("%.2f", precoAtualizado));
28
29     System.out.println("\n[TESTE 3] Atualização de Estoque (Baixa de 3 unidades do Produto 2):");
30
31     [TESTE 4] Tratamento de Exceção (Tentativa de baixa com estoque insuficiente):
32     Exceção capturada corretamente: Estoque insuficiente.
33
34     =====
35     STATUS: ARQUITETURA PRONTA PARA CAMADA WEB
36     =====
37     BUILD SUCCESS
38
39     Total time: 2.207 s
40     Finished at: 2026-09-08T19:23:52-03:00
  
```

(print do NetBeans exibindo a saída do terminal com o **BUILD SUCCESS** e os quatro testes aprovados)

Repositório Publicado no GitHub:

Link: [Mih-meromortal/pi-etapa6-refatoracao](https://github.com/Mih-meromortal/pi-etapa6-refatoracao)



Nota de Apoio Tecnológico

Este projeto e seu planejamento técnico contaram com o auxílio de ferramentas de Inteligência Artificial para apoio à produtividade durante o período pós-operatório da aluna.